Endsem Rubric

CS6.401 Software Engineering

Q1

Total Marks: 3

Rubric:

1.5 marks for correct option

1.5 marks for explanation of each correct option (0.75 for why this metric and 0.75 for how did you derive this value)

0 mark if incorrect option selected along with correct option

Answers:

Correct Option: (b)

Explanation:

(a) - No. of lines of code is 14 in the snippet, but this doesn't have to do with the decision complexity.

(b) - CC = 1+ No. of decision points = 4. Since, No. of decision points = 4; 1+4 = **5**.

Marks will be awarded even if e-n+2p (14+11+(2*1) = 5) formula is used and justified

(c) - CBO = number of distinct external classes used, hence this is not correct answer.

(d) - Cognitive Complexity = 6 , below is the explanation for the answer:

public Agent assignAgent(Order order) { if (order.getDistance() > 10.0) { // +1 if (order.getWeight() > 5.0) { // +2 return heavyLongDistanceAgent; } else { // +1 return standardLongDistanceAgent; } } else if (order.getDistance() > 3.0) { // +1 return nearbyAgent; } else if (order.isPriority()) { // +1 return expressAgent; } return defaultAgent; }

1+2+1+1+1 = **6**

Q2

Total Marks: 3

Correct Answer: b. Manage event rate and Maintain multiple copies by caching current match state and serving the clients from the cache.

- 1.5 mark for choosing option b.
- 1 mark for identifying the architectural bottleneck (e.g., extremely high read-to-write ratio, 2 million users querying the same data, relational DBs aren't meant for this level of identical concurrent reads).
- 0.5 mark for explaining how caching solves it (e.g., offloads the database, serves requests from faster memory, handles concurrent reads efficiently).

The question asks for the most appropriate tactic**.** A maximum of 1 mark to be awarded if justification has partial understanding of the caching principle.

Why a, c and d are incorrect:

(a) addresses hardware capacity, not the underlying architectural bottleneck. The problem is caused by 2 million users repeatedly querying the same data, creating an extremely high read-to-write ratio. Even with a larger database server, the database remains the single read bottleneck handling millions of identical concurrent reads. Also, since this spike is caused by a specific event (the IPL final), continuously vertically scaling infrastructure for temporary traffic surges is neither cost-effective nor operationally practical. It is not feasible to repeatedly upgrade database hardware every time concurrency spikes.

In (c) parallelizing threads still directs all requests to the same database. More threads increase contention rather than reducing it under this load pattern.

(d) reduces load by degrading service for users. This is a demand-suppression tactic. This isn't an architecture solution. It does not improve response time for the users who are permitted through.

[For q3 to q5, 0 mark if any incorrect option selected along with correct option]

Q3

Correct: (1.5 mark for selecting 1.5 mark for reasoning)

Violations: Single Responsibility Principle (SRP), Separation of concerns, GOD class

(b) Broken Modularization

Reasoning: The code/concern should exist as **separate modules/classes**, but are **collapsed into a single class**

Multiple independent concerns in same class:

- Flight path calculation
- Battery monitoring
- Weather API

- Compliance
- Logging

or

[Only (a) works for valid strong justification]

(a) **Missing Abstraction** (this is not ideal answer here as it focuses about repeated/duplicate logic)

The class contains multiple distinct concepts as code fragment not formal abstraction: Battery monitoring, Weather handling, Flight logic, Logging But none of these are modeled as separate classes/interfaces, Concepts exist only as code chunks But formal abstractions (classes/interfaces) are missing with valid assumptions.

Incorrect:

(c) Leaky Encapsulation, No evidence of exposing internal state

(d) No design smell, Clearly wrong → 4200 LOC + multiple domains

Q4

Correct: (1.5 mark for selecting 1.5 mark for reasoning)

(b) ~5.26 minutes/year; apply active redundancy with automatic failover and heartbeat-based fault detection (and reasoning).

Incorrect:

(a) 52 minutes, Wrong calculation (that's ~99.99%)

(c) Manual restart, Too slow → violates 5-minute SLA

(d) 8.7 hours, That's ~99.9%, far below requirement

Q5

Correct: (0.5 mark for selecting correct option, 0.5 for reasoning for each option)

(a) Violates low coupling, Inheritance chain tightly binds: SmartClassroom → Room → Building Changes in Building propagate through all subclasses ⇒ High coupling due to misuse of inheritance

(c) Violates high cohesion, "is-a" instead of "has-a" relationship, Classes represent mixed/incorrect concepts, Responsibilities are not well-focused

[for each option 1 mark if correct reasoning]

(d) Protected Variations, Changes in Building affect everything, No abstraction barrier

Incorrect:

(b) Information Expert, Principle: assign responsibility to the class with required data, Issue here is **wrong relationship modeling**, not responsibility allocation, Concern is “what the entities are,” not “who should handle behavior”

Q6

Total marks: 10

Part A - 3 marks

Expected Answer: Monolithic Architecture

This is be based on the evidence given context. Justification should be there for it. Atleast 2 pros and 2 cons for the architectural pattern related to this scenario should be mentioned. No marks will be allocated for the pros and cons, if they are not considering the scenario and mentioning just for the architectural pattern.

#	Criterion	Marks
1	Pattern identification and justification	1.0
2	Pros	1.0
3	Cons	1.0

Part B - 4 marks

Expected Answer: No, this architecture does not adequately handle scalability or performance for 10,000 sensors across 500 farms

The bottlenecks are structural to the chosen pattern combination, not tunable through more hardware.

A rough load estimate: assume 10,000 sensors at one reading per 30 seconds is approximately ~330 writes per second, before dashboard reads, alert evaluation, or correlated bursts (e.g., a sudden temperature drop triggering many farms simultaneously). A single synchronous-write repository will not sustain that under contention.

Valid Proposed Alternative Architecture: Combination of EDA + microservices or just EDA is also fine. Additional patterns may also include Broker pattern, pub-sub pattern with the proposed alterante patterns(EDA/microservice)

Justification for this on why and where it is applicable should be mentioned.

#	Criterion	Marks
1	Verdict with reasoning	1
2	Alternative architecture: components and diagram	1
3	Justification tied to bottlenecks	2

Note: 0 marks will be provided if the expected answer is YES and we will not consider any further explanation.

Part C - 3 marks

Expected Answer: Strategy Pattern

Farm managers need to define rules like: "if soil moisture < 20% **AND** temperature > 40°C, trigger irrigation alert"

A proper justification on why it is strategy should be given. Also if there is any other suitable pattern with correct justification is given, then marks will be provided here.

#	Criterion	Marks
1	Pattern selection	0.75
2	Justification tied to scenario	0.75
3	UML class diagram quality	1.5

Q7

Total marks: 5

Part A

Total marks: 2

Expected Answer: REJECT.

WHY? The ADR dictates that each component has "its own data store." However, it is explicitly stated that these AI/ML modules require "continuous collaboration through a shared and evolving knowledge base" where the output of one heavily influences the other. Separating the data stores would require constant, complex data synchronization via REST APIs. This leads to high latency, data duplication, and distributed consistency problems.

- 1 mark for stating Reject. 0 pts for Accept.

- Another 1 mark for correctly identifying that "database-per-service" contradicts the need for a shared, evolving knowledge base and/or mentioning the negative impact of REST API overhead/latency for highly interdependent AI modules.

Part B

Total Marks: 3

Alternative Pattern (Maximum marks obtainable: 1)

- 1 mark for proposing Blackboard, or a well-justified Event-Driven Architecture.
- OR, 0.5 pts if you propose Monolith or Layered (it solves the shared data issue but ignores the distinct computational needs of the modules).

Comparison (Maximum marks obtainable: 1)

- 0.5 mark for comparing Quality Attribute 1 (e.g., Performance, Latency, Data Consistency).
- AND, 0.5 mark for comparing Quality Attribute 2 (e.g., Modifiability, Extensibility, Scalability).

Comparisons must be contextualized to the PlaceIT scenario.

Updated ADR (Maximum marks obtainable: 1)

- 1 mark for providing a correctly formatted ADR (Context, Decision, Consequences; or any valid format) that aligns with their newly proposed alternative and mentions the final status as "Accepted".

Q8

Rubric:

Part-a [3 marks]:

Rubric:

For each correct and unique kind of technical debt [must identify at least 3; 1 mark per → 3 x 1 = 3. **Best 3 will be considered.**]:

- Type and explanation of technical debt, but no evidential justification - upto 0.5 marks max
- Type of technical debt (explanation optional) + evidence - 1 mark

Note: No marks will be awarded for just listing 3 random technical debt types without linking it to the scenario.

Part-b [3 marks]:

Rubric:

For each **distinct** ASR [must identify at least 4; 0.75 marks per → 0.75 * 4 = 3. **Best 4 will be considered**.]:

- Identification of the ASR - 0.25 marks
- Valid justification based on the scenarios in question 8(b) - 0.5 marks

Correct ASRs based on the scenarios:

- "must support 10 million customers across 3 countries": **Scalability**
- "handle real-time transactions": **Performance (major), Availability (must link it to this requirement)**
- "meet regulatory compliance and audit requirements": **Auditability/Security/Testability (or, something similar)**
- "integrate with both NorthBank and SouthBank's legacy systems during a migration": **Interoperability/Extensibility/Integrability (or, something similar)**

Note-1: No marks will be awarded if you just write random quality attributes without any justification.

Note-2: You are not eligible to receive any marks if you do not mention any quality attribute explicitly, or if you just write the 4 requirements from the question (this was a part of your Project-3 as well! You're supposed to mention them in terms of quality attributes!)

Note-3: You were brought in to design FusionCore 2.0. You must look at the 4 requirements mentioned for 2.0 in (b) itself to decide!

Part-c [4 marks]:

Rubric:

For each correct mapping of ASR to architectural style [must identify at least 3 from whatever they've written in (b); 1 mark per → 3 x 1 = 3 marks]

- Correct architectural style/idea - 0.5 marks

- Valid explanation of how the proposed architecture helps achieve the ASR - 0.5 marks

  Note: Partial/vague explanations are eligible for partial credit up to 0.25 marks.

The remaining 1 mark is to be awarded for explaining how their different ideas accounting for different ASRs can be properly integrated into one complete system. For partial/weak integration, upto 0.5 marks may be awarded.

Valid solutions:

- **To address interoperability/extensibility/migration (*main, drives the entire arch*):** Modular monolith/microservices
- **To address performance:** Event-driven (pub-sub sort of thing; asynchronous idea)
- **To address scalability:** Some sort of API gateway/load balancing, can talk about geographically distributed servers etc.

Note-1: Microservices/SoA (Interoperability) is the most important architectural decision here.

Note-2: If you just write and explain about service-oriented architecture → max upto 2.5 marks for this part (depending upon the depth of explanation, unless you also explain about geo. scaling as mentioned earlier. Remember that when proposing to use SoA, you must talk about some notion of how it is good for real-time performance (async) to be eligible for the full 2.5 marks allocated to it.) If you explain about geographic scaling as well, then you will be eligible for the full 4 marks.

Note-3: Please note that if you just write the name of the architectural style/idea and do not justify it properly, you will be graded out of 1 mark for this part. If you only mentioned the quality attributes, then you will be graded out of 0.5 marks for this part.

Q9

Rubric :

1. for each code smell [Total 2 x 1.5 = 3 marks] :
    - name of code smell - 0.25
    - exact code evidence - 0.25
    - quality attributes impacted (atleast 2 correct) - 1
2. for each design smell identified Atleast 2 design smells , can be other than your code smells as well) [Total 2 x 1.5 = 3 marks]) :
    - Name of the design smell - 0.5
    - Code Evidence and Justification together - 1
3. [Total 4 Marks] :

- Refactoring strategy for code smells of part (a) - 0.5 x 2 = 1
- Refactoring strategy for design smells of part (b) - 0.5 x 2 = 1
- Design pattern and problem it solves - 1
- sketching the key classes and interfaces involved = 1(0.5 partial for incomplete class diagram)

Probable Answers :

Code smells

- Long Method - bookRide() contains pricing, promo, DB insert, driver search/update, and SMS sending.
- Long Parameter List - bookRide() function has a really long list of parameters.
- Primitive Obsession - String vehicleType, String promoCode, String paymentMode, double fromLat, double fromLng, etc.

- Data Clumps - fromLat, fromLng, fromAddr; toLat, toLng, toAddr; userName, userPhone, userEmail.
- Switch Statements / Conditional Complexity - if (vehicleType.equals("SUV")), if (vehicleType.equals("BIKE")), if (isPool).
- Duplicated Code - two similar SMS-sending blocks.
- Large Class - RideManager does booking, pricing, DB access, driver assignment, and notification.
- Feature Envy - RideManager directly handles driver status, fare rules, promo lookup, and SMS details.

Design smells

- Missing Abstraction - no Ride, Rider, Location, Driver, Payment, Promo, or Fare abstractions.
- Broken Modularization - booking, fare calculation, promo logic, driver logic, DB logic, and SMS logic are mixed inside RideManager.
- Missing Hierarchy - vehicle-specific fare behavior is handled using if conditions instead of hierarchy/polymorphism.
- Leaky Encapsulation - SQL details, SMS API details, and DB connection details are exposed inside RideManager.
- Deficient Encapsulation - sensitive/internal details such as DB connection configuration and API key are kept directly as class members.
- Responsibility Overload / Low Cohesion - RideManager has too many unrelated responsibilities.

Design patterns

- Strategy Pattern - replace vehicle/payment/fare conditionals in fare calculation section.
- Factory Pattern - create/select the correct pricing or payment strategy based on vehicleType, isPool, or paymentMode.
- Adapter Pattern - wrap the external SMS API calls in the SMS sending section.
- Builder Pattern - construct BookRideRequest/Ride instead of using the long bookRide() parameter list.
- Observer Pattern - send notifications after driver assignment instead of direct SMS calls inside bookRide().

Bonus

Total Marks: 5

Rubric:

If 10 or more questions are correct – 5 marks

If 5–9 questions are correct – 2.5 marks

If 0–4 questions are correct – 0 marks

Answers:

1 - Concern, 2 - Views, 3 - Connectors, 4 - Models, 5 - Pipe, 6 - EDA, 7 - Refactoring, 8 - Autonomous, 9 - Serverless, 10 - Sustainability, 11 - Mediator